

Rapport — Jour 3

Automates d'arbres (pivot) & hash-consing

Binôme : DOSSOU Persévérance & AGBOGBA Silas • Modules : `formlang/tree.py`
 — Apps : `morpho`, `shield`, `hashcons`

A. Réponses théoriques

Q3.1 (0,5) — déterminisme & coût

Dans `morpho_automaton` et `shield_automaton`, les règles sont enregistrées dans un dictionnaire `delta` indexé par la clé (`symbole`, `états_enfants`). Un dictionnaire Python associe au plus une valeur à une clé donnée : il ne peut donc jamais exister deux règles $f(q_1, \dots, q_n) \rightarrow q$ et $f(q_1, \dots, q_n) \rightarrow q'$ avec $q \neq q'$ pour la même clé — les deux automates sont **déterministes** par construction. *Conséquence* : pour un terme t , `TreeAutomaton.run` calcule un unique état par nœud (aucun choix, aucun retour en arrière) ; le parcours post-ordre visite chaque nœud exactement une fois $\Rightarrow$ coût $O(n)$ où n est le nombre de nœuds de t .

Q3.2 (0,5) — la frontière ne suffit pas

Deux arbres $T_1 = \text{word}(\text{prefixes}(\text{prefix}(\text{na}), \text{nil}), \text{rest}(\text{root}(\text{pend}), \text{nil}))$ (classe `PREFIXED`, frontière `na` `pend`) et T_2 où le même préfixe `na` serait rattaché comme second niveau d'un circonfixe (structure imbriquée différente, même séquence de feuilles `na` `pend`) ont une frontière identique mais des classes différentes (`PREFIXED` vs. `CIRCUMFIXED`, cf. énoncé §4.2.3). Un AFD de mots ne lit que la *séquence* des feuilles, jamais leur regroupement en sous-arbres ; il ne peut donc pas distinguer T_1 de T_2 , alors que le BUTA lit l'arbre *en profondeur* (règles `prefixes/rest`) et les classe différemment.

Q3.3 (0,5) — théorème du yield

Un mot morphologique accepté est de la forme `word(pc, rest(root, sc))` où `pc` est une chaîne `prefixes` de $p \geq 0$ préfixes (état `PFCHAIN` ou `NIL`) et `sc` une chaîne `suffixes` de $s \geq 0$ suffixes. La frontière (yield) est la concaténation, dans l'ordre de lecture gauche-droite, des labels des feuilles `prefix`, `root`, `suffix` : exactement $p^* r s^*$ (un préfixe répété, une racine, un suffixe répété) — c'est *le même langage régulier* $p^* r s^*$ que celui de l'AFD du Jour 1 (§2, avec p, r, s jouant le rôle de a, o, r). Le BUTA généralise donc bien l'automate fini : sa frontière retombe sur le langage régulier déjà reconnu par le DFA/NFA du Jour 1.

Q3.4 (0,5) — reduplication

La reduplication $\{ww \mid w \in \Sigma^*\}$ n'est pas un langage d'arbres régulier. Un BUTA ne dispose que d'un nombre fini d'états Q ; pour accepter ww , l'automate devrait, en lisant la seconde moitié, « comparer » chaque lettre à la lettre correspondante de la première moitié déjà lue — or l'état courant ne peut encoder qu'une information bornée (au plus $|Q|$ possibilités), indépendante de $|w|$. Pour $|w|$ suffisant, deux mots distincts $w \neq w'$ de même longueur mèneraient au même état après lecture de la première moitié, et l'automate ne pourrait plus distinguer ww de ww' : un argument de pompage sur les états (analogue au lemme de l'étoile) montre l'absence de reconnaissance. Culy (1985) illustre précisément ce phénomène pour le bambara, dont le vocabulaire dépasse le hors-contexte à cause d'un motif de reduplication.

B. Exercices de programmation

E3.1

`TreeAutomaton.run` : étiquetage post-ordre récursif (état REJECT propagé dès qu'un enfant rejette ou qu'aucune règle ne correspond); `accepts` teste l'appartenance de l'état racine à `self.final`. Tests `test_morpho_*`, etc. débloqués.

E3.2

Règles Δ de `morpho_automaton` : feuilles `prefix/root/suffix/nil`, chaînage `prefixes/suffixes`, `rest` (racine + suffixes), `word` (préfixes + rest) $\rightarrow$ WORD. Tests `test_morpho_accept_et_classes`, `test_morpho_rejet` : passés.

E3.3

`_seq(x,y)` : DANGER si l'un des deux est déjà DANGER ou si $\{x,y\} = \{\text{OVR}, \text{ROLE}\}$, sinon la sévérité maximale; `frame/sys` font passer un OVR ou ROLE directement encapsulé à DANGER. Test `test_shield_blocage` : passé.

E3.4

`product(a1,a2)` : états = paires (q_1, q_2) , une règle $(sym, (q_1, q_2) \dots) \rightarrow (r_1, r_2)$ pour chaque paire de règles de même arité sur le même symbole; `finaux` = $F_1 \times F_2$. Test `test_produit_intersection` : passé.

E3.5

`CompactStore.intern` : internement récursif des enfants, clé canonique (`symbol`, `label`, `kids_ids`), partage via table; `get` reconstruit l'arbre exact (`round-trip`); `compression` = $1 - \text{uniques}/\text{total}$. Tests `test_round_trip_exact`, `test_sous_arbres_identiques_partages`, `test_compression_positive` : passés.

E3.6 (bonus)

`enc_automaton` : compte les feuilles `enc` plafonné à 2 (même signature que `shield_automaton`); `dangerous_and_double_encoded` = `product(shield_automaton(), enc_automaton())`. Test `test_shield_double_encodage_P45` : passé.

Sortie de référence obtenue

```
$ pytest -q tests/test_tree.py tests/test_hashcons.py
8 passed
```